WebGL을 이용한 Sierpinski Carpet 구현

컴퓨터그래픽스 과제 보고서

분할 횟수 제어 · RGB 색상 제어 · 재귀 기반 도형 생성

작성일: 2026년 9월 27일

1. 과제 개요

이 프로젝트는 WebGL과 재귀 알고리즘을 사용하여 Sierpinski Carpet을 생성한다. 사용자가 분할 횟수와 RGB 색상값을 변경하면 결과가 즉시 갱신되도록 구현하였다. 화면에는 과제에서 요구한 입력 요소와 렌더링 결과만 배치하였다.

2. 이론적 배경

Sierpinski Carpet은 정사각형을 가로와 세로 방향으로 각각 3등분한 뒤, 가운데 정사각형을 제외한 8개의 정사각형에 같은 과정을 반복하여 생성하는 프랙탈이다. 분할 횟수가 d일 때 최종 사각형 수는 8^d이다.

WebGL은 JavaScript에서 준비한 정점 데이터를 GPU 버퍼에 저장하고, 정점 셰이더와 단편 셰이더를 거쳐 화면에 도형을 그린다. 정점 셰이더는 입력 좌표를 클립 좌표로 전달하며, 단편 셰이더는 uniform으로 받은 하나의 색상을 모든 단편에 적용한다.

3. 프로그램 설계

프로그램은 index.html, style.css, app.js로 구성하였다. HTML은 600×600 캔버스, 분할 횟수 슬라이더, R·G·B 숫자 입력 필드를 제공한다. JavaScript는 재귀적으로 정점을 생성하고 WebGL 버퍼에 업로드한다. GLSL 셰이더는 정점의 위치와 출력 색상을 처리한다.

처리 순서는 다음과 같다.
1) WebGL 컨텍스트 생성
2) 정점·단편 셰이더 컴파일 및 프로그램 연결
3) 재귀 호출로 최종 사각형 계산
4) 각 사각형을 두 삼각형으로 변환
5) Float32Array를 GPU 버퍼에 업로드
6) gl.drawArrays로 렌더링
7) 입력 변경 시 필요한 값만 갱신

4. 재귀 분할 구현

divideSquare(x, y, size, depth)는 현재 정사각형의 왼쪽 아래 좌표, 한 변의 길이, 남은 분할 횟수를 입력받는다. depth가 0이면 현재 정사각형을 최종 도형에 추가한다. 그렇지 않으면 한 변을 3등분하고, 행과 열이 모두 1인 가운데 영역을 건너뛴 뒤 나머지 8개 자식에 대해 함수를 다시 호출한다.

```
childSize = size / 3
childX = x + col × childSize
childY = y + row × childSize
```

각 재귀 호출에서 depth를 1씩 줄이므로 반드시 종료 조건에 도달한다. 부모의 시작 좌표에 자식 크기의 배수를 더하는 방식으로 모든 자식의 위치를 계산한다.

5. WebGL 렌더링

WebGL의 기본 요소로 삼각형을 사용하였다. 사각형 하나는 두 삼각형으로 나누며, 인덱스 버퍼를 사용하지 않으므로 사각형당 여섯 개의 정점을 저장한다. 생성된 배열은 Float32Array로 변환한 뒤 gl.bufferData로 GPU에 전달한다.

정점 셰이더의 attribute vec2 aPosition은 2차원 좌표를 받고 gl_Position에 전달한다. 단편 셰이더는 uniform vec4 uColor를 받아 도형 전체에 같은 색상을 적용한다. 캔버스의 배경은 gl.clearColor로 지정하고 실제 도형은 gl.drawArrays(gl.TRIANGLES, ...)로 그린다.

6. 사용자 입력 처리

분할 횟수가 바뀌면 사각형의 위치와 개수가 달라지므로 정점 배열을 다시 생성하여 GPU 버퍼를 갱신한다. 색상 변경은 기하 데이터에 영향을 주지 않으므로 배열을 다시 만들지 않고 단편 셰이더의 uniform만 변경한다.

R, G, B 입력값은 각각 0-255 범위로 제한한다. 입력값을 255로 나누어 WebGL의 0.0-1.0 범위로 변환하고 gl.uniform4f를 사용하여 전달한다.

7. 실행 결과 및 검증

분할 횟수별 예상 사각형 수는 8^d이며, 사각형 하나가 6개의 정점을 사용하므로 예상 정점 수는 6×8^d이다.

분할 횟수	예상 사각형 수	예상 정점 수
0	1	6
1	8	48
2	64	384
3	512	3,072
4	4,096	24,576
5	32,768	196,608

분할 횟수 1, 3, 5에서 가운데 영역이 반복적으로 제거되는지 확인하였다. 정점 배열의 길이를 이용하여 계산한 정점 수도 표의 예상값과 일치했다. RGB 값 변경 시 정점 데이터는 유지되고 화면의 도형 색상만 즉시 변경되었다.

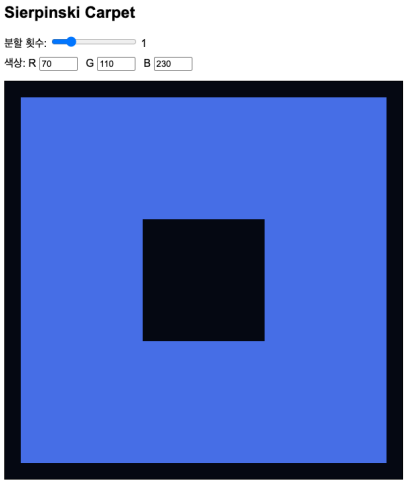


그림 1. 분할 횟수 1

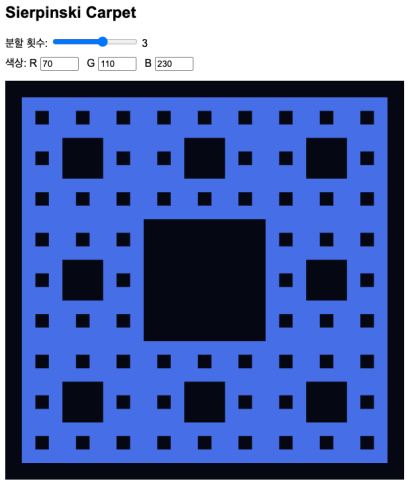


그림 2. 분할 횟수 3

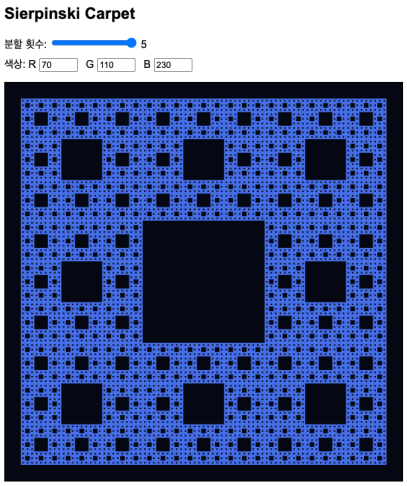


그림 3. 분할 횟수 5

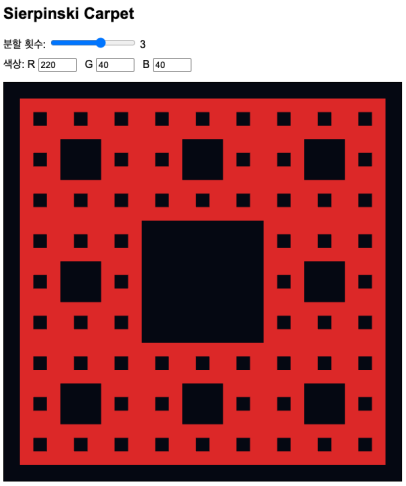


그림 4. RGB 색상 변경

8. 문제점과 해결 과정

부모 정사각형의 한 변 길이와 좌표값을 처음에는 혼동하였다. 예를 들어 왼쪽 아래 좌표가 (2, 5)이고 한 변이 6인 정사각형의 오른쪽 위 좌표는 (8, 11)이다. 8과 11은 길이가 아니라 위치를 나타내는 좌표이며 각 축의 좌표 차이가 6이다. 이 구분을 바탕으로 자식 좌표를 부모의 시작 위치와 자식 크기의 배수로 계산하였다.

분할 횟수를 변경할 때 이전 정점 뒤에 새 정점이 누적되지 않도록 정점 배열을 먼저 초기화하였다. 또한 색상만 바뀔 때는 불필요하게 기하 데이터를 다시 생성하지 않고 uniform만 갱신하도록 역할을 분리하였다.

9. 결론

재귀 분할로 Sierpinski Carpet의 기하 데이터를 생성하고 이를 WebGL 버퍼와 셰이더를 사용하여 렌더링하였다. 분할 횟수 변경은 기하 데이터를 재생성하고 색상 변경은 uniform만 갱신하도록 구성하였다. 이를 통해 JavaScript에서 만든 데이터가 GPU 버퍼와 셰이더를 거쳐 화면에 표시되는 WebGL의 기본 처리 과정을 확인하였다.

10. 소스 코드

GitHub 저장소:
https://github.com/stone0min/webgl-sierpinski-carpet

11. AI 도구 사용

JavaScript의 배열, 함수, 반복문과 재귀 개념을 학습하고 Sierpinski Carpet의 좌표 계산 및 WebGL 렌더링 구조를 이해하기 위해 AI 도구를 사용하였다. 부모 정사각형의 위치와 길이 구분, 3×3 분할 시 자식 좌표 계산, 재귀 종료 조건, WebGL 버퍼와 셰이더의 역할에 관한 설명을 받았다. 또한 프로젝트의 초기 구조와 보고서 초안을 만드는 데 도움을 받았다. 생성된 내용은 직접 실행하고 각 함수의 동작과 결과를 검토한 뒤 수정하여 사용하였다.

12. 참고 자료

- 수업 자료: 컴퓨터 그래픽스 개요, 그래픽스 프로그래밍 1·2, 입력 및 상호 작용
- Edward Angel, Dave Shreiner, Interactive Computer Graphics: A Top-Down Approach with WebGL